

# Exercice 1 — Dockerfile multi-stage

Thème : construction d'image, build multi-stage, réduction du footprint

---

## Contexte

On dispose d'une application Java (Spring Boot) construite avec Maven, produisant un fichier exécutable `app.jar` dans le dossier `target/`.

## Objectifs

- Écrire un Dockerfile en deux étapes (build puis run).
- Comprendre pourquoi séparer la compilation de l'exécution réduit la taille de l'image.

## Travail demandé

- 1 Écrivez la **première étape** du Dockerfile, nommée `build`, qui part d'une image contenant Maven et le JDK, copie le code source, et exécute la commande de packaging Maven pour produire `app.jar`.
- 2 Écrivez la **seconde étape**, qui part d'une image légère contenant uniquement un JRE, et qui copie **seulement** le fichier `app.jar` produit à l'étape précédente (grâce à `COPY --from=build`).
- 3 Définissez le point d'entrée (`ENTRYPOINT`) du conteneur pour démarrer l'application avec `java -jar app.jar`.
- 4 Exposez le port 8080 utilisé par l'application (`EXPOSE`).
- 5 Expliquez, en deux ou trois phrases, pourquoi l'image finale ne contient ni Maven ni le JDK complet, et pourquoi c'est un avantage en production.

## Critères d'évaluation

- Le Dockerfile contient bien deux étapes distinctes, clairement nommées.
- Seul l'artefact compilé est copié dans l'image finale, pas le code source ni les outils de build.
- L'explication sur la réduction de taille est correcte.